

Abstract—The Adobe Creative Jam for North America focuses on project creation utilizing Adobe’s Premiere Rush. The final product for submission in the contest is a short film utilizing Rush, to be submitted and judged by Adobe. The target of this proposal is to collect successful film projects, compare these projects, and discover patterns that help propel projects to success, then using these patterns determine a strategy and structured plan for the project’s submission. Here is the repository link and information: https://github.com/aceydufresne/cs4412-Plotting_Plots-Success, and <https://www.kaggle.com/datasets/thedevastator/rotten-tomatoes-top-movies-ratings-and-technical>.

Specifically searching for what successful films contain, that less successful films do not, or at least contain less of, essentially the disjunction of the set of success and the set of non-successful. The final output, or result for this pattern searching should be the attributes and aspects of cinema that affect the outcome the most significantly. In this way, we should be able to create a rough ‘script’/outline for the contest, or a very abstracted prediction of what the most potentially successful project we could build.

I. INTRODUCTION

Two datasets were used for this project, the first was [Rotten Tomatoes dataset](#), from an independent [Kaggle user](#)^{1.1}. The dataset contains over nine-hundred entries, ranging from films made in 1920 through 2020, a large range, and quite recent. This data set has a plethora of information that can be utilized, including the critical ratings, audience ratings, and financial return.

This data set is primarily used to determine the results, how successful a project was, having multiple metrics of success is important as well. For example, if a film is very financially successful, but struggles in ratings, both critical and otherwise, then we can plot the ‘result,’ on a graph with a vector embedding. Counter wise, if a film has top notch reviews, but was financially unsuccessful, we can compare the vector values of both, taking into account all aspects of what makes a project well built.

However, this data set only addresses the results of a project. To find out why a film is successful, we need to know what a film is. Quite blatantly, we may say a film is determined by a budget, or perhaps the crew and actors that worked on that project. Here we run into an issue, how can we determine how much of an impact each component has on the final result. How can we know how much of the film was

impacted by the director, or the camera crew lead, or perhaps the main cast. How much does the budget affect the final output? Are these even what makes a film, a “film”?

‘The Grand Budapest Hotel,’ and ‘I, Frankenstein’ were both released in the same year, With Wes Anderson’s film having half the budget Stuart Beattie’s project had access to, and yet Anderson almost doubled the box office returns of ‘I, Frankenstein.’ Surely, this is not the right metric, with outliers like this, which we can show by comparing the films in the [data set](#).¹¹ against one another.

The audience may be influenced when going to theatres, or browsing a streaming service, by who directed a movie, or the cast that acts within it, but rarely do audiences make choices based on budget reports. Another flaw, just because a director makes a good film, does not mean they will always perform at this standard, in fact this is a rare feat, with only a few exceptions. The counter example, of notably seasoned and skilled directors who typically, over their career, build successful franchises and films, sometimes make ill quality films.

We don’t necessarily need to put too much of an emphasis on these factors as well, what we really need is to find the patterns of successful films, based on those films alone, not who made them. So how do we determine this? Well, when we watch a movie, the medium is visual, quite apparently a good film looks good. A movie should be compelling, the story should keep viewers engaged. How do we create metrics for this type of non-qualitative standards. Obviously as humans we know when a film looks good, the lighting is deliberate, the color grading is pleasing to the eye, the individual shots are framed well.

This means we need a new data set, a data set of films, the average film that a viewer watches in the theatre is typically shot at twenty-four frames per second, with theatre releases having an average run-time between ninety minutes and one-hundred-twenty minutes. That’s five-thousand-four-hundred seconds for a ninety minute movie, and one-hundred-twenty-nine-thousand-six-hundred frames, with over nine-hundred movies in our initial dataset, that comes out to be 116,640,000 frames for the complete dataset.

That’s enough to determine a lot of patterns, but we run into some issues. Memory is a big concern, for this dataset I have an external SSD with 2-terabytes of storage, this is not enough to hold the proposed

dataset. We can get around this issue, if we think of a movie, the average movie will have a lot of shots filmed on a rig, so many shots will be similar to one another. We just need a general idea of how the ‘ambience’ is within a film, so we can use a SNN, to compare frames and if two frames have above a threshold value of similarity we only need to keep one.

The next problem is a little more complicated, how do we gather over nine-hundred movies. Surely streaming services, for example, Amazon, Netflix, and Disney, have all these entries. This is the case, but all these services have preventative measures to deter obtaining copies of these films. Which means we need to manually screen record all these films. With nine-hundred movies, each around an hour and a half long in duration, this is well over a thousand hours of recording. Nearly two months of solid recording, with no breaks between. I don’t have the time or resources to facilitate this process.

So we need a custom dataset, with frames from these movies, but we can’t directly access the films, or rather, we choose not to. Unfortunately, there is no public data set that has what we are looking for, is there? If you notice from the original dataset that was introduced, the data set is based on information from the corporation ‘Rotten Tomatoes,’ but not released by them. We can follow the same process for a custom data set. ‘ImDb,’ is another movie critic site, with a plethora of data, including over nine-hundred images per the average film. This is more than enough for what the project calls for.

The first step is to create an agent that can remotely access the ‘ImDb’ site, it then iterates through the initial data set, and saves all the images for each film. The script is run using a combination of the PlayWright library, and the BeautifulSoup library. At the same time, the agent also saves the Synopsis of the films, which as you may recall the initial dataset also contains, but a slightly different version. This will become important in a few moments.

The other important aspect of a movie is the plot. As we collect the synopsis of films, we are building our corpus to decompose these narratives. We now have two summarizations of each movie, and can begin to compare the elements of a narrative that make a movie good.

As the agent is building this data set, we can begin to plot and vectorize the data, before it is complete. This is the benefit of utilizing the first data set, we already have the results. So, essentially we can manually back-propagate to find what are the most important aspects of a film. As we find values for each component we are searching for, we can compare to how the film performed financially, and critically, then begin to determine which aspects influence the

results the most.

II. ARCHITECTURE AND STRUCTURE:

The flow of the pattern determination: the main program initializes the main dataset, and the agent. The agent begins scraping data from the ‘ImDb site.’ Passing that data into a combined data set. Which can then be used to form

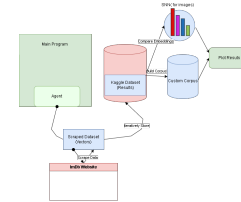


Figure 1. Rough Architecture Planned

the vectors which will be analyzed in the ‘Algorithms and Techniques,’ section. As you can see, there is a proposed SNN, which would be used essentially to create the embeddings for the frames, and then use the algorithms mentioned to find the distance between vectors.

III. ALGORITHMS AND TECHNIQUES

The output from our initial efforts will be two distinct vector values, one for images, broken up between contrast(color grading), brightness(lightning), framing rules(rule of thirds). Measuring these gets very tricky, and we’ll try multiple methods out, but the proposed method for without testing, is to use Cosine Similarity. This way the magnitude doesn’t overpower the rest of the metrics. For example, two films can have very different light levels, but can feel visually similar because of their color grading, or contrast.

Right now we have color grading and contrast grouped together, for simplicity, but they are different. Color grading is the culmination of all the colors appearing in a frame, so we can use a histogram vector: which is a collection of histograms, with each histogram representing a specific primary color. Contrast on the other hand: we convert the frame from RGB to grey scale, and use the formula: $(\text{Max} - \text{Min}) / (\text{Max} + \text{Min})$, and store the results in another histogram vector. Additionally, we can find the Mahalanobis distance between the frame attributes, for a base reference. We want to use this over Euclidean distance, because Mahalanobis is good for outlier detection, which we are specifically looking for, as outliers may represent the film aspects that are the cause of success.

For the text based summation of movie plots, we tokenize the text, and then will use Cosine Similarity, I won’t go into the specifics, as I am taking Natural

Language Processing right now, so I am still learning how to apply the NLP algorithms.

As discussed previously, the scale will be an issue, we talked about budgets, while I do not believe they are overtly important to the end outcome, we still need to check, just in case, so all the datasets need to be normalized, to see what attributes are not contributing anything to the end results. Likewise, with as many dimensions as we plan to measure, budget, contrast, lighting, color-grading, cast, director, year-made, framing, plot-narrative, we will most certainly have issues with scaling, which is why we are using the Cosine distance, to attempt to combat this, as well, we will most likely apply some dimensionality reduction, especially after normalization, we may have a better understanding of what we don't need to check for.

As discussed previously, the scale will be an issue, we talked about budgets, while I do not believe they are overtly important to the end outcome, we still need to check, just in case, so all the datasets need to be normalized, to see what attributes are not contributing anything to the end results. Likewise, with as many dimensions as we plan to measure, budget, contrast, lighting, color-grading, cast, director, year-made, framing, plot-narrative, we will most certainly have issues with scaling, which is why we are using the Cosine distance, to attempt to combat this, as well, we will most likely apply some dimensionality reduction, especially after normalization, we may have a better understanding of what we don't need to check for.

This is where the Apriori Algorithm may become useful, as long as we normalize the data, and prepare it a little more, since we know the results of each film before hand, we can take only the successes, and apply labels to our attributes, so for example, everything has a one-hundred percent amount possible, so for the rule-of-thirds, one-hundred percent would look like three rectangles stacked ontop of one another, then we can apply a percentage of how close the input is to that maximum value. To do this, we need to decide what classifies a successful film, for example top ten percent of box offices returns, any film that is in this top margin would qualify, or maybe the top twenty-five percent and above a seventy-five percent in critic reviews. We can run multiple tests, for each case scenerio, testing which ones return the best results.

IV. DISCOVERY QUESTIONS

We've already went over the questions we need to answer, but to briefly summarize:

- What attributes are the most important, or have the largest impact on the output of a film's success.

- How much does the brand of a movie, cast and directors, affect a film, compared to the visual aspects of cinema.

- Can a studio "buy" success, we have several examples that they cannot, but removing outliers, is there a pattern where: "more money = more general success."

The final prediction should just be a set of ranked attributes, in order of prevalence. For example, what we aren't looking for: the most successful movie, we want: what unique factors and attributes contributed to the general overall success of each film. As we already know what movies performed well, we need to move backwards from success, to determine the function that created that said success.

V. ROUGH SCHEDULE:

The timeline is a bit rough, as the to create the proposal I had to begin the initial implementation:

Feb 5. Turn in Proposal, M1	Feb 6.	Feb 7.	Feb 8. Work on rough implementation	Feb 9. Finish rough implementation
Feb 10.	Feb 11.	Feb 12. Work on applying techniques and algorithms	Feb 13. Finish and run algorithms, begin plotting data	Feb 14. Work on M3, final implementation, start writing the final thesis.

Figure 2. Rough Calendar

The web scraper is already built, so I just need to run it, and collect the data, I do need to take several days, to work on several other projects, which if you have time to check out:

- [monocular depth estimation algorithm, dataset preparation, that we discussed last semester](#)
- [Completely ethical LinkedIn career data scraper\(does not violate any terms and conditions, I'm sure\)](#)

Part II: Implementation:

VI. INITIAL IMPLEMENTATION:

In the pre-processing portion of this paper, we covered a rough plan to process and analyze the data of two datasets. In the previous section we discussed "What attributes are the most important, or have the largest impact on the output of a film's success," one area we predicted may have a heavy hand in success, was the plot. To determine this, we first need to re-process the data, to get a baseline idea of how plots can affect success.

A. Pre-processing:

To get the base case in our example, we can plot the data based on genre, with the assumption that genre's are more likely to contain similar stories. To do this, we first need to sort the data set into separate groups, based on genre. Then we can plot the genres as box-plots:

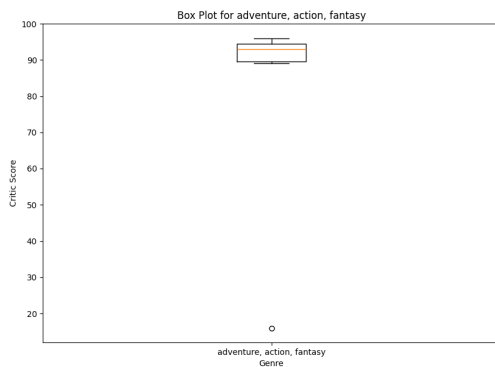


Figure 3. Initial boxplot.

These results look very good, on the surface, it seems that genre is a good determination for a film's success, with only one outlier, and every other film above ninety percent! However, if we continue forward: So what is happening here? Upon closer inspection, the dataset is too organized. What we mean here, is that critics are very intricate with their dissection of cinema, meaning a movie may have several genres, such as this example, where a genre: fantasy, animation, action, kids and adventure, may only contain a few, or even, in this case, one film. To solve this we need to do even more processing, starting with a new organization of the dataset. So now the program uses a nested for loop, to iterate through the sub genres of a movie's genre. This leads to a movie having several genres, potentially. Is this an issue?

Un-equivocally not, a film can have multiple genres, and to exclude from one genre, will impact the accuracy of our comparison in the next step. With

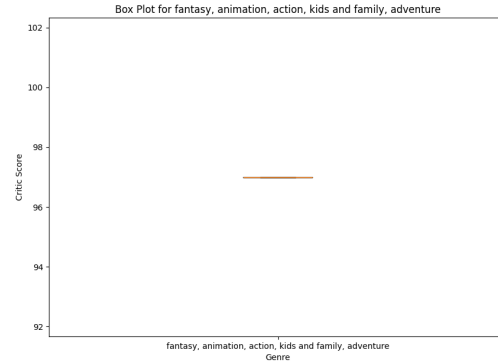


Figure 4. Bad results.

the films sorted into broader categories, each category now has at least ten films contained within it. Here is a comparison between the previously shown fantasy, with minimal processing, and the new processed fantasy genre: The results still look bright, but now

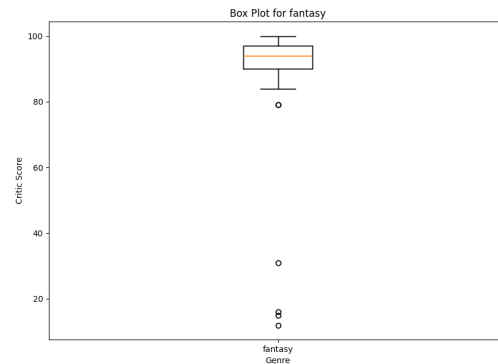


Figure 5. Updated Fantasy Genre.

we can see the average rating has dipped, as well as the addition of several outliers. This is a much more accurate portrayal of the genres ratings.

First, we wanted to find the patterns between critically acclaimed films and their plots, based on a synopsis from IMDb, and compare between films, both successful and less successful. The plan was to use cosine similarity, as cosine similarity works well with data with a lot of dimensions and features. From my Natural Language Processing Course, I was familiar with the 'Sentence Transformers' module, so the initial plan was to implement that library, with a function that compares all the films.

To implement the "Rotten Tomatoes' Top Movie," dataset, the Kaggle Python module was used, with the Pandas library for data processing. Then the program uses a map set, mapping each title to its synopsis. Then finally a brute-force technique was used to compare each film's plot against every other plot.

This was all housed in a single function, with a nested loop, that iterates through the dataset, of one-thousand films, in its entirety. On each vector we are applying a manual dot product between movie A, and movie B, then dividing by the absolute value of each.

In practice, using a brute-force technique became un-scalable, with a big O complexity of around $(n(n-1))/2$, computing all these values gave more information than was desired. For this paper, we are focusing on what patterns lead to successful cinema, so comparing a film with a review rating of seventeen percent, and one with a rating of eighty percent, isn't very enlightening. Similar to Apriori's Bottleneck, with problems of Candidate Generation and Multiple Database Scans,

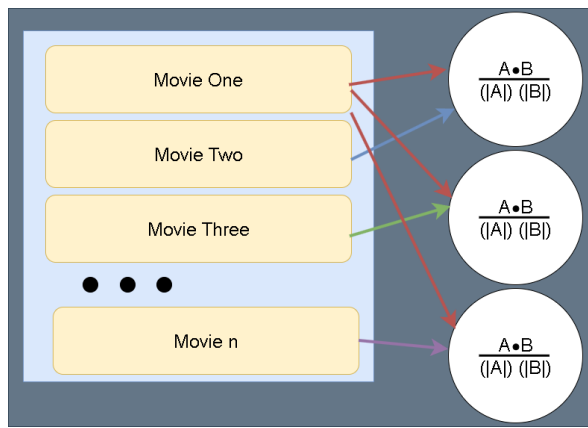


Figure 6. Function for comparing embedded plot vectors between film entries.

VII. REFINEMENT:

In the textbook, a suggested answer to these problems was FP-Growth, which was not very useful in this scenario, as the situation was not exactly the same. Instead I came up with two theoretical solutions, the first was to re-write the program to work in a multi-thread environment, specifically creating a Parallel program. The benefits of this approach would be to speed up the time complexity, not big-O complexity, we could keep all the data results, including the data we might not need, but may be useful for further data analysis. The problems, Python is not a naturally conducive programming language for Parallelism, meaning I would most likely need to either mine the data and apply pre-processing within a Python program, and return a global variable, that a Java, or C++ program can then input and perform the calculations on.

The other proposed solution was to use a nearest neighbor technique, after some reading in the course text, I used the idea of K-Fold Cross Validation, specifically splitting the initial dataset on a specific attribute, in this case the Critic Review Score. As

the text suggested, I used ten folds, separating the dataset by ten percent intervals. Then ran the cosine similarity between the vectors of each fold. This is still big $O(n^2)$ or $(n(n-1))/2$.

However, we can make further changes in the program, such as using libraries to compute the heavy lifting. Both the numpy and sklearn modules, utilize parallelism built into their libraries, so this increases the speed of computations. Furthermore, for the final implementation, I want to build a more sincere nearest neighbor algorithm, ANN, which would only calculate the lowest sets, or folds, and the highest, for example, only calculating films with less than twenty percent, and higher than twenty percent. It is difficult to design a more efficient algorithm for these patterns, as we're interested in outliers, so we need to compare as much as we can.

Once we have updated our functions to match the accuracy of our initial predictions, we can plot the results, by genre:

I decided to go with PCA, principle component analysis, which is used to reduce dimensions when plotting results. Now that I've switched over to sentence transformers, sklearn, and numpy, instead of manually computing everything. All these modules are closer to industry standards, which lets them intertwine and work together much more efficiently, as can be seen in the previous figure, disappointingly, there doesn't seem to be much correlation between genres, and there specific film plots, or embedded vectors.

However, we can go further, as mentioned prior, instead of comparing between genre categories, we can use a different attribute, the critic's score, itself. The pattern here is much more prevalent to the hypothesis, as before, what we were really asking was: "Does the genre affect the film's success?," but instead we want to know if two similar plots, will both rank highly in critic reviews. Here are some examples of those findings: (('Black Panther', 'Spider-Man: Homecoming')np.float32(0.4029472)), This has a forty percent match, which as humans we can clearly understand, both are comic book adaptations, this was the highest matching vector for 'Black-Panther,' and